

SLM BENCHMARK LAB

Local LLM Reliability & Performance Benchmark

Technical Documentation Report

Phase 1: Inference Benchmarking • Phase 2: Structured Output Engineering • Phase 3: Model Comparison Study

Prepared by Hassan Bin Abid

AI Engineer & Educator

August 2026

Table of Contents

Executive Summary..... 3

Problem Statement..... 4

Phase 1 — Inference Benchmarking..... 5

Phase 2 — Structured Output Engineering..... 6

Phase 3 — Model Comparison Study..... 7

Model Selection Guide..... 10

Quantization Analysis..... 11

Key Engineering Learnings..... 13

Executive Summary

This report documents the engineering work behind SLM Benchmark Lab, a portfolio project measuring how small language models perform when run entirely offline. Three models — llama3.2:3b, phi4-mini, and mistral:7b — were benchmarked on CPU-only consumer hardware across inference speed, structured-output reliability, and multi-category quality, then exposed through a production-style FastAPI service.

The goal was to replace opinion with evidence: rather than assuming which model or configuration is "best," every claim in this report is backed by a measured result.

Results at a Glance

Question	Answer
Fastest model	llama3.2:3b — 14.55 tok/s
Highest quality	mistral:7b — 2.92 / 3.0
Best speed/quality trade-off	llama3.2:3b — nearly 2x faster, only 0.07 below best quality
Cold vs. warm start	Cold TTFT is 2.4x higher (8,314 ms vs 3,400 ms)
Structured output reliability	Temp 0.0 → 1/5 success; Temp 0.7 + retry → 5/5 success

Problem Statement

Sending data to cloud-based LLM APIs is not always viable. Four constraints commonly rule it out in production:

- Privacy regulations — GDPR, HIPAA, and similar frameworks restrict sending sensitive data to third-party services.
- Latency requirements — network round trips add 200 ms–2 s of overhead, unacceptable in real-time applications.
- Cost at scale — API pricing compounds quickly; processing 1 million tokens per day can cost thousands of dollars monthly.
- Edge deployment — IoT devices, air-gapped systems, and remote deployments may have no reliable internet connectivity.

This project measures what running small models entirely offline actually costs in speed and quality — with evidence, not guesswork.

Technology Stack

Component	Technology	Purpose
Model Runtime	Ollama 0.20.5	Hosts and serves local LLM models
Models	llama3.2:3b · phi4-mini · mistral:7b	3B–7B parameter models for inference
Language	Python 3.11	Scripting and orchestration
Validation	Pydantic 2.12.5	Schema enforcement and validation
API Framework	FastAPI 0.135.3	REST API wrapper around Ollama
Terminal UI	Rich 14.3.3	Formatted tables and progress display
Operating System	Windows 11 — CPU only	No GPU acceleration

Phase 1 — Inference Benchmarking

Before building any application logic, we measure. This phase establishes a performance baseline for llama3.2:3b on the target hardware. These numbers inform every downstream engineering decision — model selection, timeout configuration, UX design, and cost estimation.

Metrics Captured

- Time to First Token (TTFT) — milliseconds from sending the prompt to receiving the first token. This is the latency a user perceives as a thinking pause.
- Total Response Latency — wall-clock seconds from request sent to full response received.
- Tokens Per Second — generation throughput, calculated as `output_tokens / total_latency_s`.

Summary Statistics

Metric	Average	P95
Time to First Token	3,813 ms	8,314 ms
Total Latency	21.4 s	36.0 s
Tokens Per Second	13.0 tok/s	—
Peak Throughput	17.12 tok/s	(prompt 4)
Hardware	CPU-only, Windows 11	No GPU

Engineering Observations

Cold Start vs. Warm TTFT

Prompt 1 recorded 8,314 ms TTFT — the model loading from disk into RAM. After this first call the model remains loaded and TTFT stabilizes at 3,300–3,600 ms. In production this is solved with a warmup call at server startup so no user ever feels the cold-start penalty.

Latency Scales Linearly with Output Length

Total latency is directly proportional to output token count. Unbounded generation means unbounded latency. Any production system using this model must enforce a `max_tokens` limit to guarantee SLA compliance.

Phase 2 — Structured Output Engineering

Raw LLM output is an unstructured string. Production systems almost always require structured data — validated JSON objects with typed fields that downstream code can process reliably. This phase implements the constraint → validate → retry pattern used in real AI pipelines.

The Three-Layer Pattern

Layer	Mechanism	What It Does
1	Constrained prompting	Instructs the model to return only raw JSON with specified fields, types, and constraints, including a worked example.
2	Pydantic validation	Parses and validates the response against a strict schema. Rejects wrong types, missing fields, and invalid enum values.
3	Retry with context	On failure, re-prompts once with the exact validation error included. If the second attempt fails, logs and moves on gracefully.

Temperature Experiment Results

Temperature	Successes	Finding
0.0 — deterministic	1 / 5	Markdown wrapping caused JSON parse failures
0.7 — stochastic	5 / 5	Retry mechanism recovered all failures

Key Finding — Temperature 0.0 Reduces Instruction Following

At temperature 0.0, llama3.2:3b defaulted to wrapping its JSON in markdown code blocks in 4 out of 5 cases. At temperature 0.7, added randomness caused the model to skip the wrapper and return raw JSON directly, and the retry mechanism recovered all remaining failures. For small models, temperature 0.0 can reduce instruction-following reliability — always test structured generation across multiple temperature values.

Phase 3 — Model Comparison Study

Three models were evaluated on 40 standardized prompts across five task categories: factual recall, reasoning, code generation, creative writing, and instruction following. All models ran on the same CPU-only Windows 11 hardware with temperature=0.0 and seed=42 for reproducibility.

Models Under Test

Model	Parameters	Size on Disk	Character
llama3.2:3b	3B	2.0 GB	Fast, lightweight baseline
phi4-mini	3.8B	2.5 GB	Microsoft efficiency-focused model
mistral:7b	7B	4.4 GB	Larger model, higher quality target

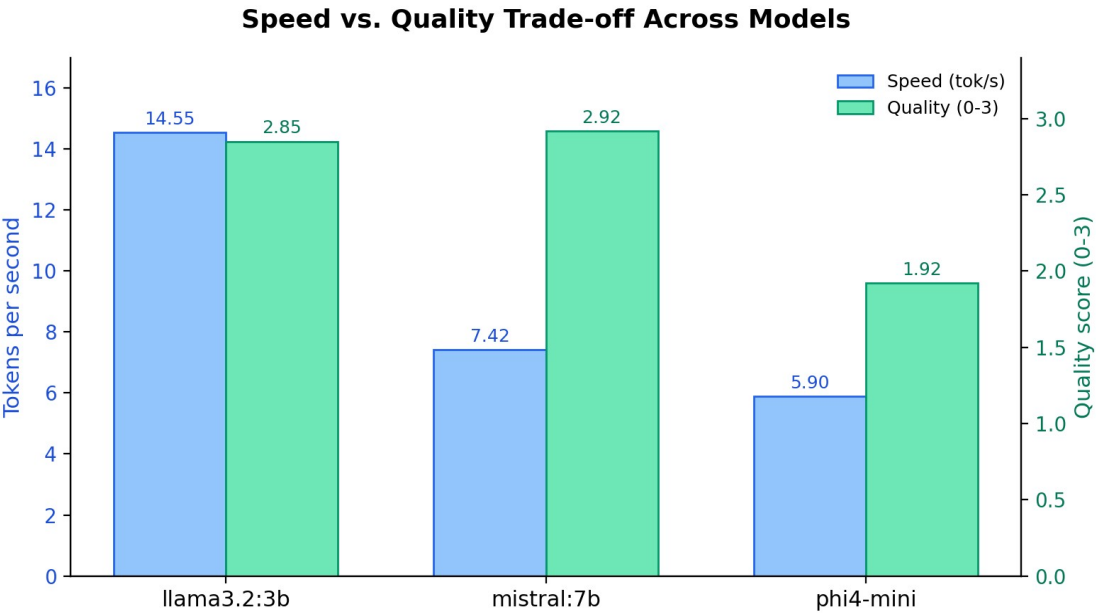


Figure 1 — Throughput (tok/s) vs. overall quality score (0-3) for all three models.

Speed Comparison — All Models

Model	Avg Tok/s	Avg TTFT (ms)	Avg Latency (s)	Avg Word Count
llama3.2:3b	14.55	3,149	9.8	116
mistral:7b	7.42	3,452	23.6	119
phi4-mini	5.90	3,575	85.2	772

Quality Scores — All Models

Model	Overall	Factual	Reasoning	Code	Instruction
mistral:7b	2.92 ★	3.00	2.88	3.00	2.75
llama3.2:3b	2.85	3.00	2.50	2.75	3.00
phi4-mini	1.92	3.00	1.25	1.00	1.62

Scale: 3 = correct, complete, well structured | 2 = correct but incomplete | 1 = wrong or garbled

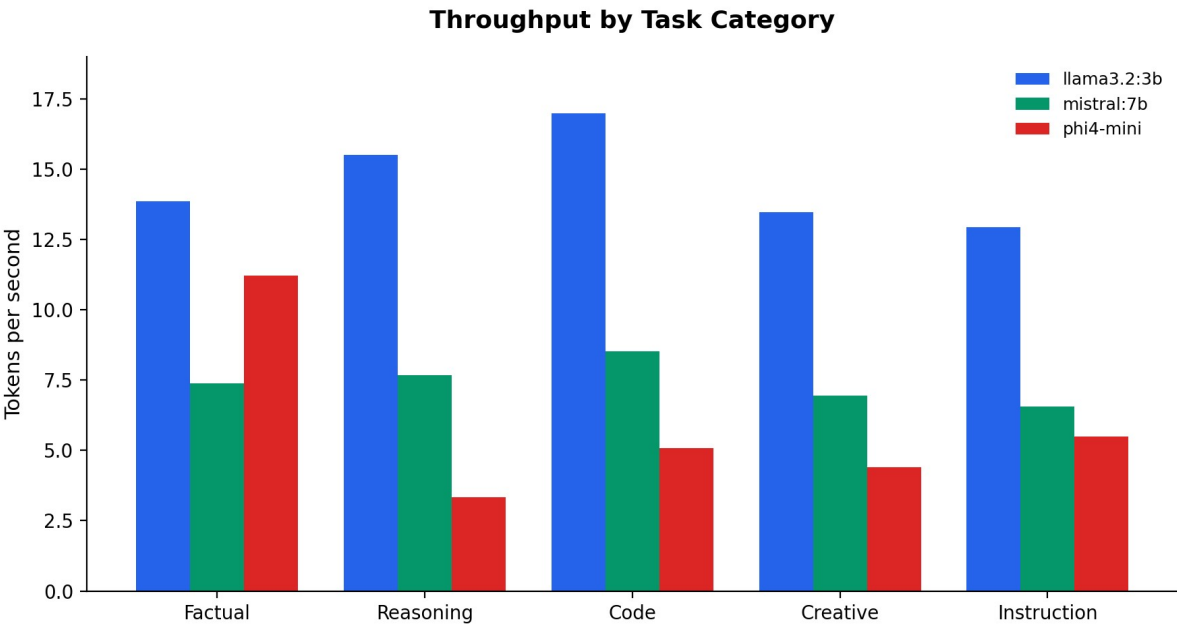


Figure 2 — Tokens per second by task category. llama3.2:3b leads every category.

Key Findings

Finding 1 — mistral:7b is the best overall model

mistral:7b achieved the highest quality score of 2.92/3.0, with perfect scores on factual, code generation, and creative writing. Its larger 7B parameter count translates directly into better reasoning (2.88 vs 2.50 for llama). The trade-off is speed — 7.42 tok/s vs llama's 14.55 tok/s — and memory, requiring 4.4 GB vs 2.0 GB.

Finding 2 — phi4-mini's verbosity does not equal quality

phi4-mini generated an average of 772 words per response — 6.6x more than the other models — yet achieved the lowest quality score of 1.92/3.0. More output does not equal more correct output.

Finding 3 — llama3.2:3b is the best speed-quality trade-off

llama3.2:3b achieved a quality score of 2.85 — only 0.07 below mistral:7b — while running at nearly 2x the speed. For latency-sensitive applications, it is the clear choice, and it also uses the least memory at 2.0 GB.

Finding 4 — All models perform equally on factual recall

All three models scored 3.0/3.0 on factual recall. Model size only matters for generative, reasoning-heavy, or format-following tasks — not for factual retrieval where the answer is directly encoded in training data.

Model Selection Guide

Practical guidance for choosing a model given specific deployment constraints:

Deployment Constraint	Recommended Model	Reason
Latency critical (<5s response)	llama3.2:3b	14.55 tok/s — fastest by 2x
Best quality, speed secondary	mistral:7b	2.92/3.0 quality score
Limited RAM (<4 GB available)	llama3.2:3b	Only 2.0 GB disk, lowest memory
Code generation tasks	mistral:7b	Perfect 3.0/3.0 on code category
Instruction following tasks	llama3.2:3b	3.0/3.0 on instruction category
Avoid phi4-mini when	Precision required	1.92/3.0 overall — verbose but inaccurate

Quantization Analysis

After completing the three-phase benchmark, a post-hoc investigation was conducted into the quantization format of all locally pulled Ollama models, revealing a significant and non-obvious engineering finding.

Investigation Method

Ollama exposes an `/api/show` endpoint that returns model metadata including quantization level, format, parameter count, and internal digest ID. All locally pulled models were queried and their digest IDs compared.

Model Inventory and Quantization Details

Model	Quantization	Format	Parameters	Size on Disk
llama3.2:3b	Q4_K_M	gguf	3.2B	2.0 GB
llama3.2:3b-instruct-q4_K_M	Q4_K_M	gguf	3.2B	2.0 GB
mistral:7b	Q4_K_M	gguf	7.2B	4.4 GB
mistral:7b-instruct-v0.3-q4_K_M	Q4_K_M	gguf	7.2B	4.4 GB
phi4-mini	Q4_K_M	gguf	3.8B	2.5 GB

Key Finding — Ollama Already Distributes Q4_K_M by Default

Every model pulled via Ollama is already distributed as GGUF Q4_K_M format. The digest IDs confirmed this conclusively: llama3.2:3b and llama3.2:3b-instruct-q4_K_M share an identical digest ID — they are the same file on disk. The entire Phase 1 and Phase 3 benchmark results already reflect Q4_K_M quantized performance.

Full Precision vs. Quantized — Memory Comparison

Model	Q4_K_M Actual	F32 Estimated	Memory Saved
llama3.2:3b	2.0 GB	~12 GB	6x less memory
mistral:7b	4.4 GB	~28 GB	6x less memory

Quantization Format Comparison

Format	Bits/Weight	Quality Kept	vs F32 Memory	Best Use Case
F32	32 bits	100% baseline	1x	Research only
F16	16 bits	~99.9%	0.5x	GPU with VRAM
Q8	8 bits	~99.5%	0.25x	High quality CPU
Q4_K_M	4 bits	~96-98%	0.13x	CPU default
Q2	2 bits	~80-85%	0.06x	Extreme constraints

Production Recommendations

- CPU-only edge deployment — always use Q4_K_M. Ollama provides this automatically, giving near full-precision quality at a fraction of memory cost.
- GPU deployment with ample VRAM — consider F16 for marginally better quality on reasoning-heavy or code generation tasks.
- Extreme memory constraints (under 2 GB RAM) — Q2 is viable for factual recall only; avoid for code generation or multi-step reasoning.
- Never use F32 on CPU — the memory and speed cost is prohibitive with negligible quality benefit over Q4_K_M.

Key Engineering Learnings — All Phases

Finding	Production Implication
Cold start TTFT is 2.4x higher than warm	Always warm up model at server startup — never let a user feel first-load latency
Throughput is 16–17 tok/s on CPU (llama)	Set max_tokens limits to cap worst-case latency for SLA compliance
Temperature 0.0 reduced JSON compliance	Test structured generation at multiple temperatures — 0.0 is not always safest for small models
Retry with error context achieved 100% recovery	Always include the exact validation error in retry prompts
Pydantic caught all malformed responses	Never pass raw LLM output into application logic — always validate at the perimeter
mistral:7b best quality at 2.92/3.0	Larger models justify their resource cost for quality-critical tasks
phi4-mini: 6.6x more words, lower quality	Verbosity is not a proxy for quality — evaluate correctness, not length
All models equal on factual recall	Model size only matters for generative and reasoning tasks, not memorized facts
llama3.2:3b best speed-quality trade-off	For edge/latency-constrained deployment, llama3.2:3b delivers near-mistral quality at 2x the speed

End of Report — SLM Benchmark Lab • Phases 1, 2 and 3